

Rekursion und dynamische Programmierung

Aufgabe 1

Schreibt eine Funktion `fib(n)`, die die n -te Fibonaccizahl zurückgibt. Erstellt eine Liste der ersten 10, 20 und 50 Fibonaccizahlen.

Erstellt eine Liste `fib` = [1, 1], in der ihr die folgenden Fibonaccifolge einlagert. Nun könnt ihr die letzten beiden Elemente der Liste nutzen, um die darauffolgenden Folgenglieder zu berechnen.

Hinweis: Die Folge der Fibonaccizahlen wird rekursiv definiert als $f_0 = f_1 = 1$ und $f_n = f_{n-2} + f_{n-1}$.

Für die folgenden Aufgaben benötigt ihr eine Liste an Primzahlen. Ansonsten dauert die Berechnung unnötig lang. Diese findet ihr im Moodle zum Download.

Folgendermaßen könnt ihr die Datei dann einbinden:

```
with open("primes.csv") as f:
    primes = []
    for line in f:
        for prime in line.split(", "):
            primes.append(int(prime))
```

Danach könnt ihr mit der Liste `primes` arbeiten. Diese enthält nun alle Primzahlen bis 10 000.

Aufgabe 2

Die Goldbachsche Vermutung besagt, dass jede gerade Zahl größer als 2 Summe zweier Primzahlen ist. Prüft diese Vermutung für alle geraden Zahlen bis 10 000.

Aufgabe 3

Die schwache Goldbachsche Vermutung besagt, dass jede ungerade Zahl größer als 5 die Summe dreier Primzahlen ist. Überprüft auch diese Vermutung.

Aufgabe 4

Nochmal zurück zur Goldbachschen Vermutung: Wie viele Möglichkeiten gibt es, die Zahl 8 000 als Summe zweier Primzahlen darzustellen? Wie sieht es bei den anderen Zahlen aus? Stellt die Anzahl der Möglichkeiten, eine gerade Zahl als Summe zweier Primzahlen darzustellen, in einem Scatterplot dar.

Dafür braucht ihr ein Paket:

```
import matplotlib.pyplot as plt
```

Dann müsst ihr eure Daten aufbereiten. Dafür empfehle ich zwei Listen:

```
x = [4, 6, 8, ...]  
y = [f(4), f(6), f(8), ...]
```

Hierbei steht x für die Zahl, die ihr als Summe zweier Primzahlen darstellen wollt und $f(x)$ die Anzahl der Möglichkeiten, die ihr dafür habt. Anschließend könnt ihr das Scatterplot zeichnen lassen.

```
plt.scatter(x, y)
```

(Mit matplotlib kann man noch ganz andere Plots darstellen. Eine Übersicht dazu findet ihr im Netz!)



This document is subject to the Creative Commons Zero (CC0) License.
To create this document, we used L^AT_EX.

Christians Social Media Kanäle:

BeReal: <https://bere.al/cspannagel>

Bluesky: <https://bsky.app/profile/cspannagel.bsky.social>

Discord: <https://tinyurl.com/chrisp-discord>

Facebook: <https://www.facebook.com/chr.spannagel>

GitHub: <https://github.com/dunkelmunkel>

Instagram: <https://www.instagram.com/dunkelmunkel/>

LinkedIn: <https://www.linkedin.com/in/christian-spannagel-31b7b4a1/>

Mastodon: <https://scholar.social/@cspannagel>

Snapchat: <https://www.snapchat.com/add/spannagelc>

Telegram: <https://t.me/cspannagel>

Tellonym: <https://tellonym.me/christian.spannagel>

TikTok: https://www.tiktok.com/@_cspannagel_

Threads: <https://www.threads.net/@dunkelmunkel>

Twitch: <https://www.twitch.tv/cspannagel>

X/Twitter: <https://twitter.com/dunkelmunkel>

Youtube: <https://www.youtube.com/pharithmetik>